🌐 Module 05: Web Application Penetration Testing

```
# Exercise 11: Extracting the Database Schema and List of User Credentials via SQL Injection
```

Scenario

In this lab, we perform UNION-based SQL injection attacks against the OWASP Juice Shop application to extract database schema definitions and dump user credential tables.

SQL Injection (SQLi) occurs when untrusted user input is directly concatenated into a backend database query without adequate sanitization or parameterization. Attackers can leverage SQLi to bypass authentication, read arbitrary database tables, and extract sensitive information such as password hashes and API secrets.

Objective

The objective of this lab is to help students learn how to:
- Identify SQL Injection vulnerabilities in RESTful API endpoints (`/rest/products/search?q=`) using Burp Suite Repeater
- Determine query column count using `UNION SELECT` statement column-matching techniques
- Enumerate SQLite database schema via the `sqlite_master` table
- Extract user accounts, role definitions, and password hashes from the backend `Users` table

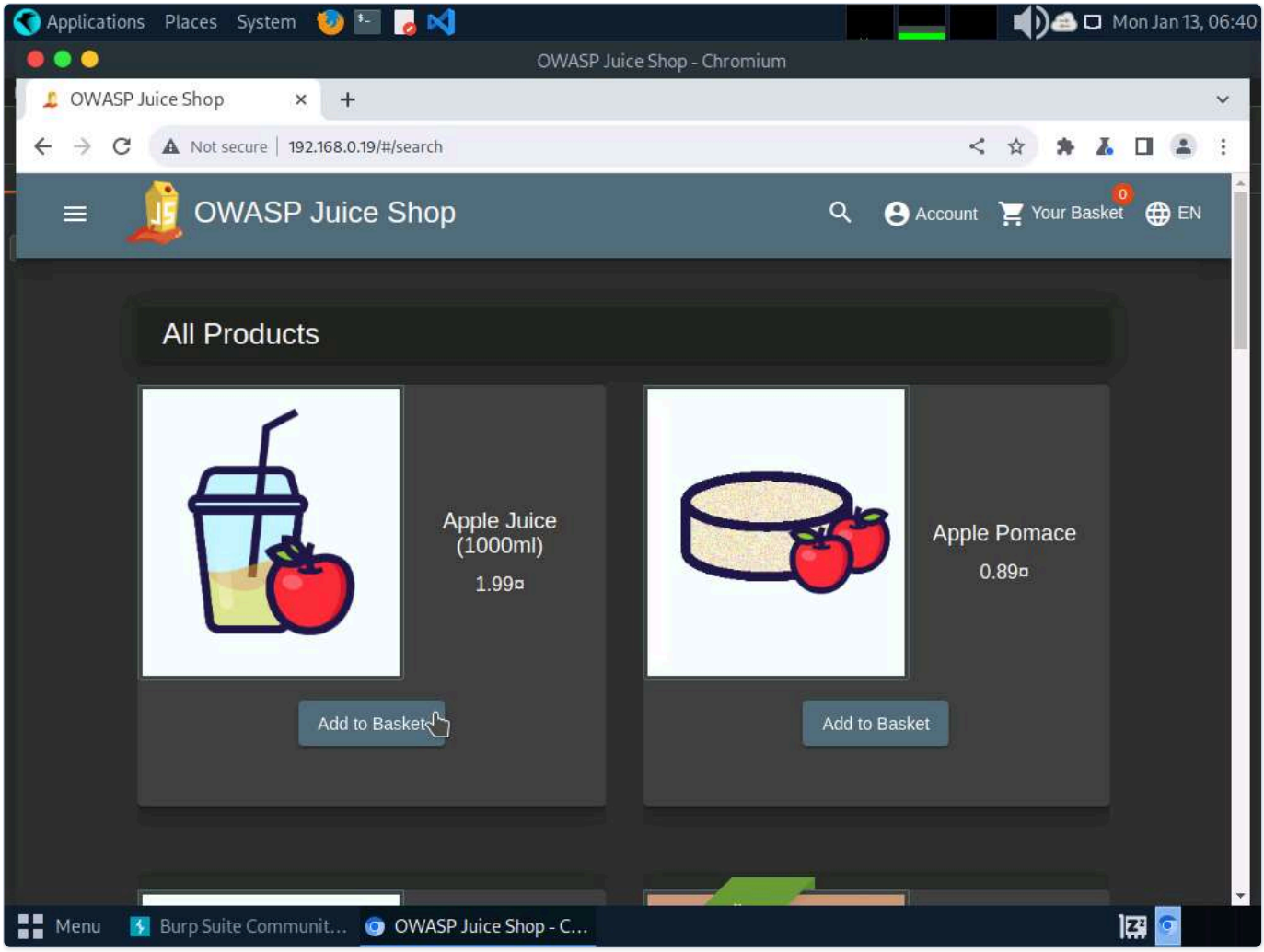
---

Task 1: Extracting the Database Schema and List of User Credentials via SQL Injection

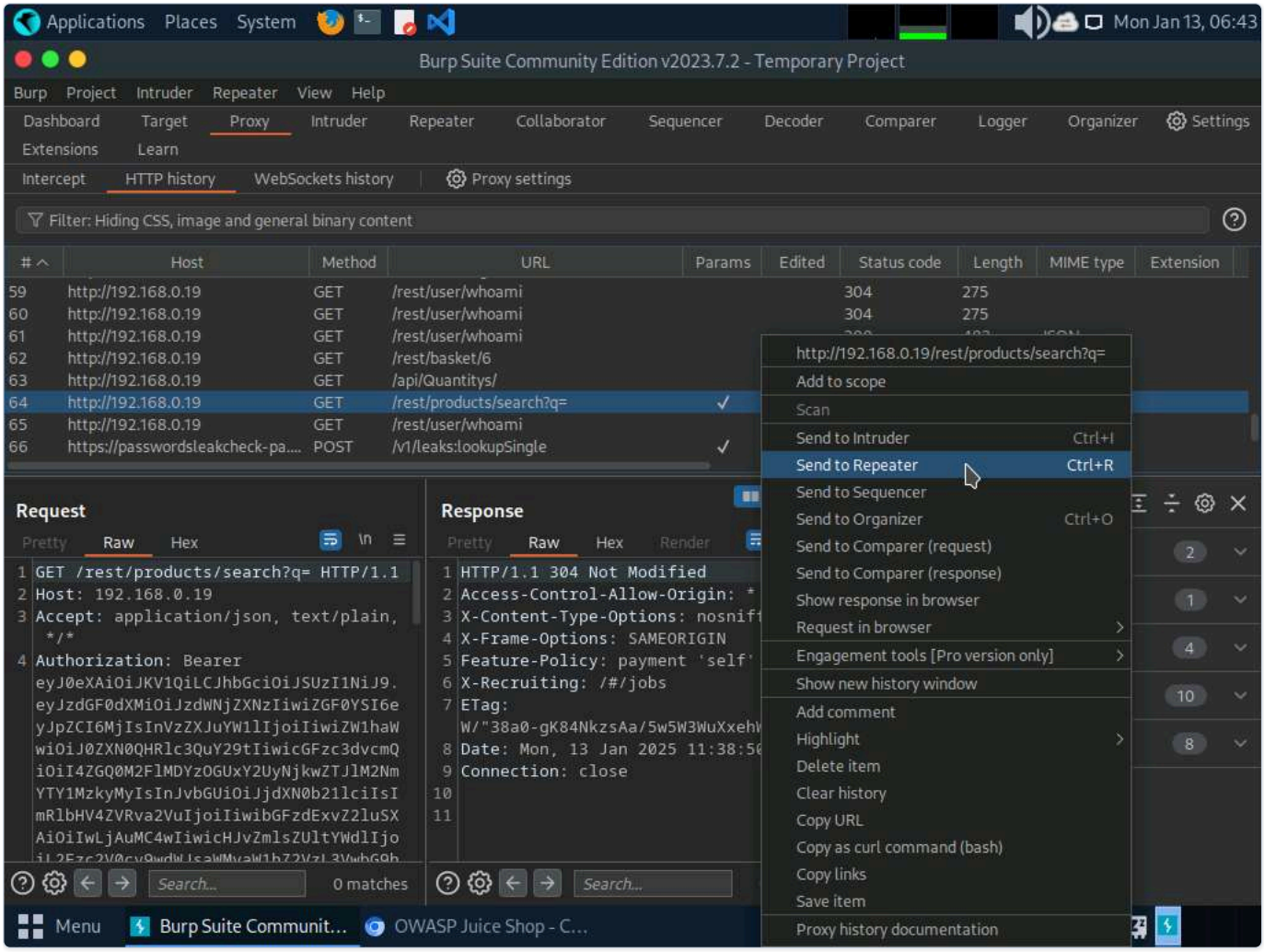
In this task, we launch the OWASP Juice Shop container, capture product search requests via Burp Suite Proxy, manipulate search parameters in Repeater to trigger SQLite syntax errors, determine column positioning, extract table schemas from `sqlite_master`, and dump all rows from the `Users` table.

Lab Tasks

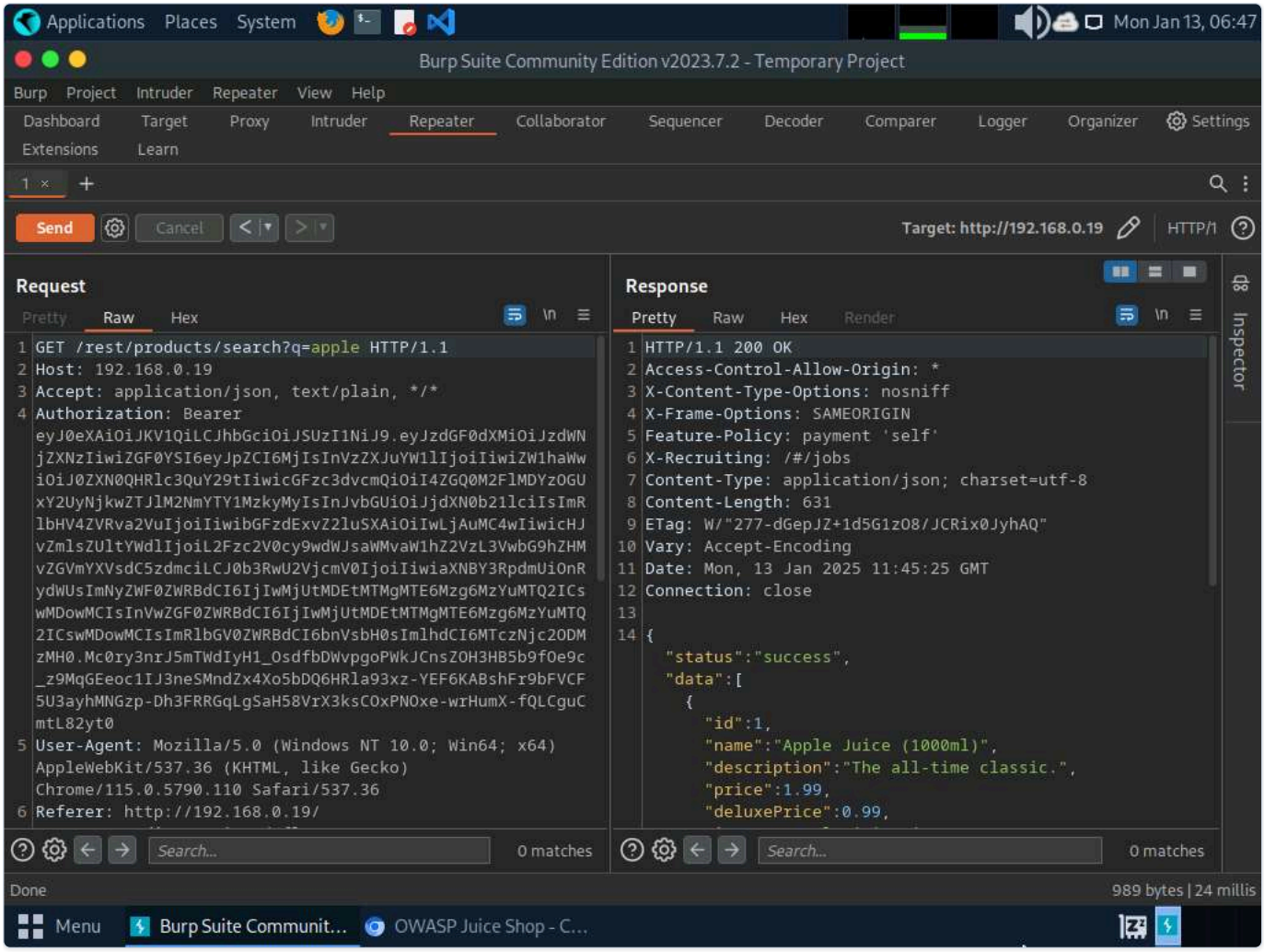
1. Switch to the **Ubuntu** machine, type **toor** in the Password field, and press Enter.
2. Open a terminal with sudo privileges and launch the Juice Shop Docker container:
   sudo docker run -d -p 80:3000 bkimminich/juice-shop
3. *(If an Apache port conflict error occurs, execute **sudo service apache2 stop** first.)*
4. Switch to the **Parrot Security** machine and log in with credentials **pentester** / **toor**.
5. Launch **Burp Suite**, navigate to the **Proxy** tab, and click **Open browser**.
6. In Burp's embedded browser, navigate to **http://192.168.0.19**.
7. *(Follow steps #9-11 from Exercise 05.5 to initialize Burp Suite if needed).*
8. Log into your OWASP Juice Shop account (created in Exercise 05.5) and add Apple Juice to the basket by clicking **Add to Basket**.



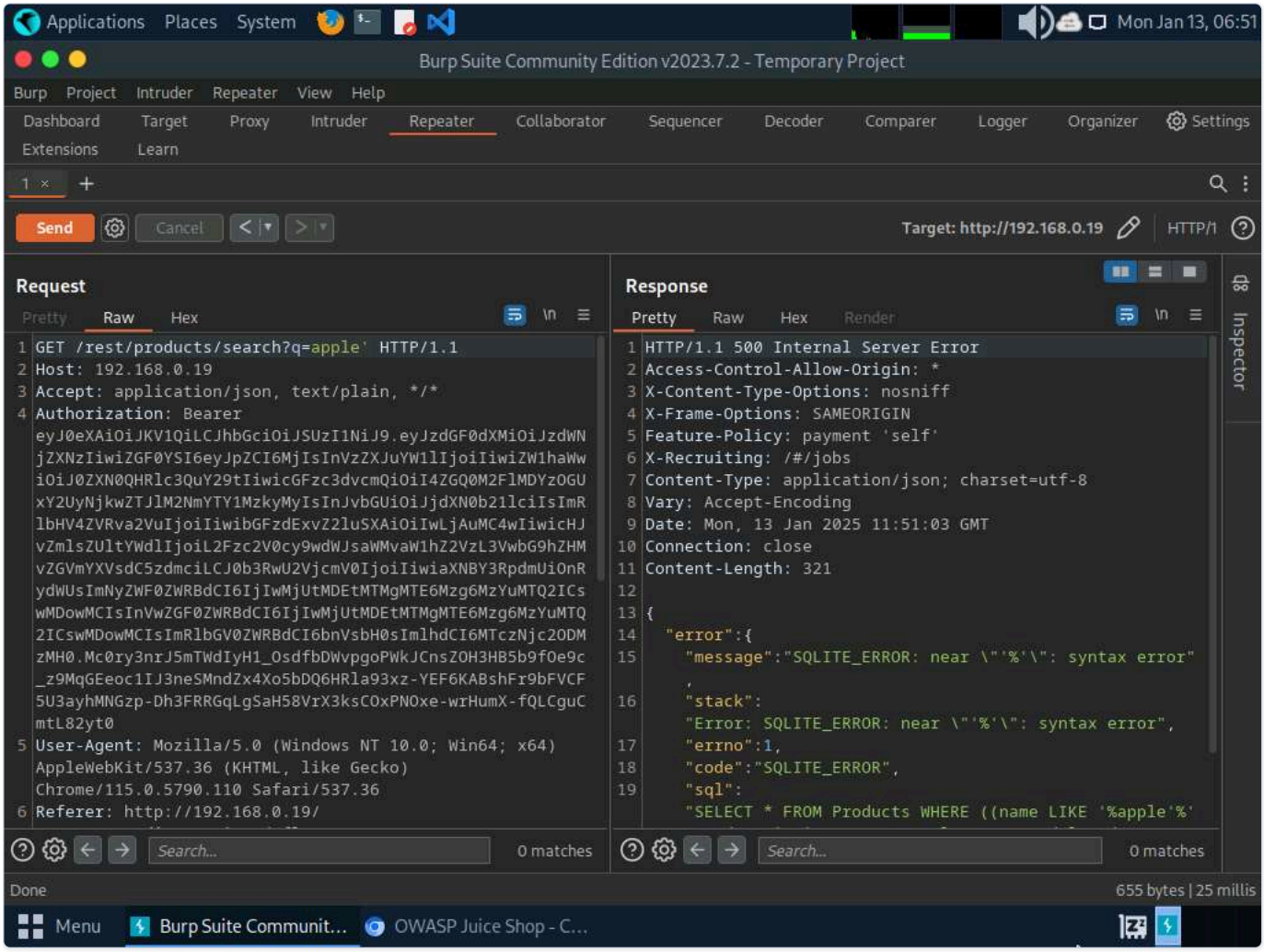
1. Switch to the **Proxy** tab in Burp Suite, turn **Intercept is off**, and click **HTTP history**. Locate the **GET /rest/products/search?q=** request (Status 200/304), right-click it, and select **Send to Repeater**.



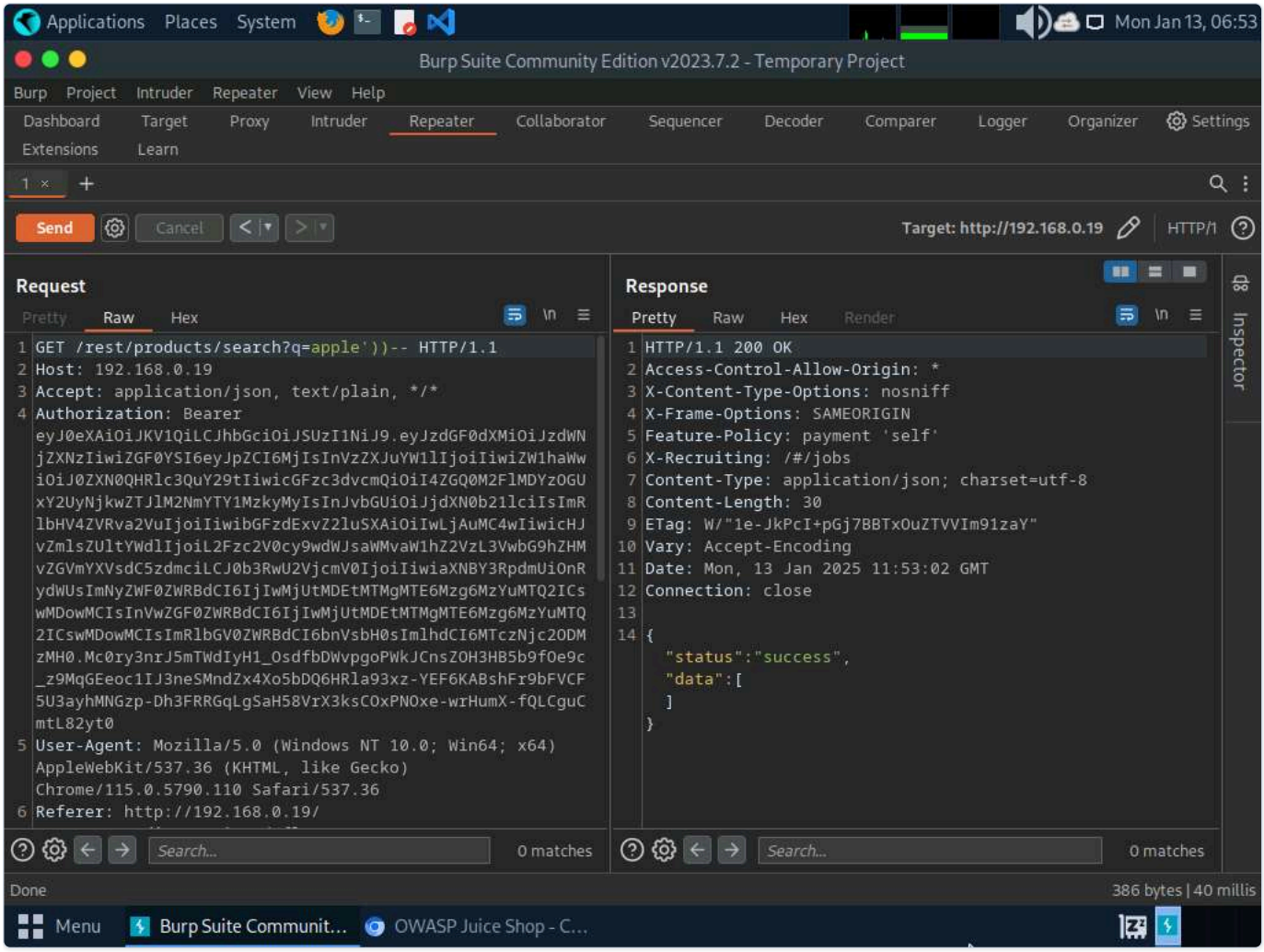
1. Switch to the **Repeater** tab, ensure **q=apple** is present in the request query line, and click **Send**.



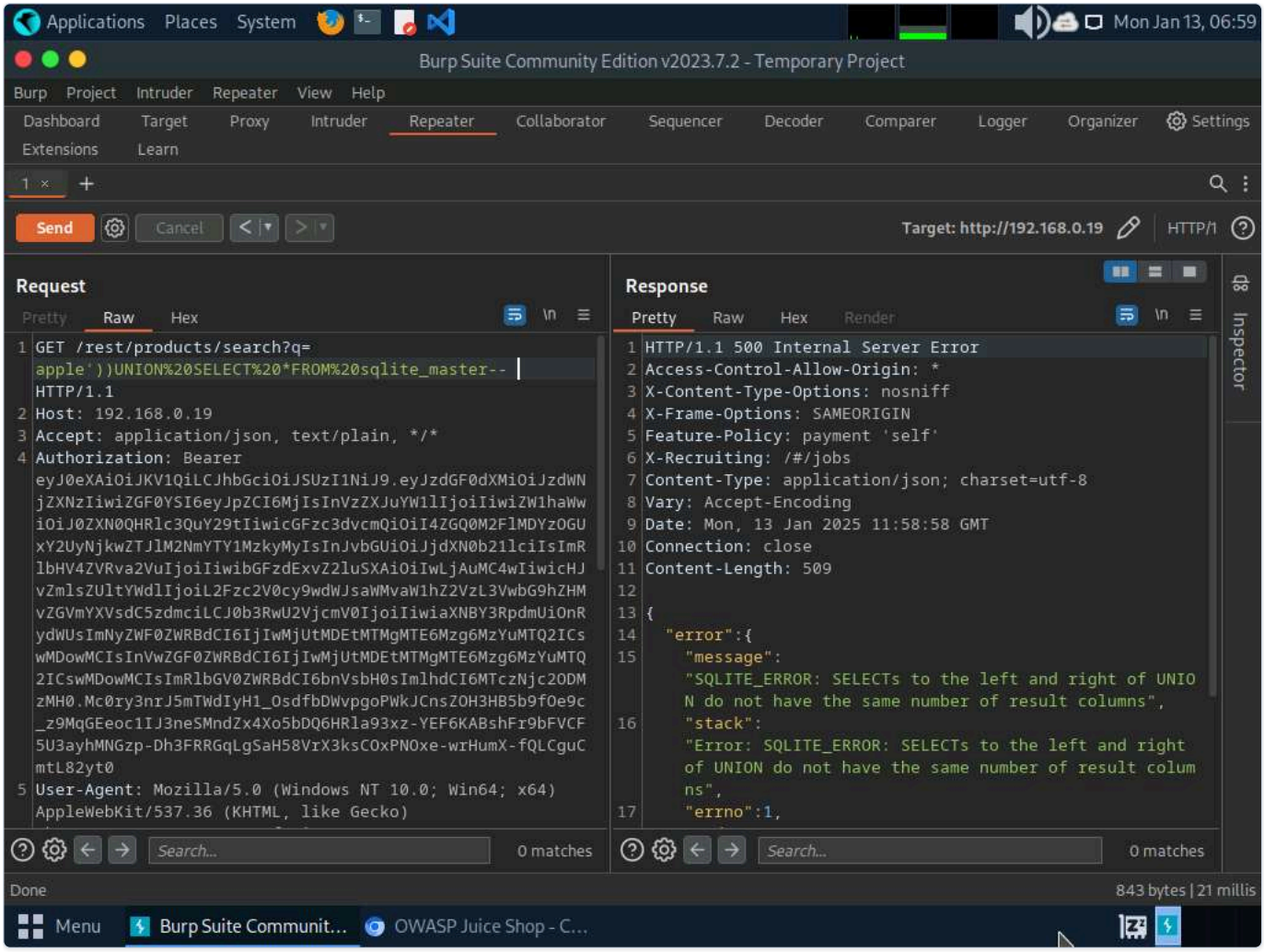
1. Append a single quote (') to the search parameter value (`q=apple'`) to test for SQL syntax errors. Click **Send** and observe the **SQLITE_ERROR** message in the response pane, confirming an unhandled SQL injection vector.



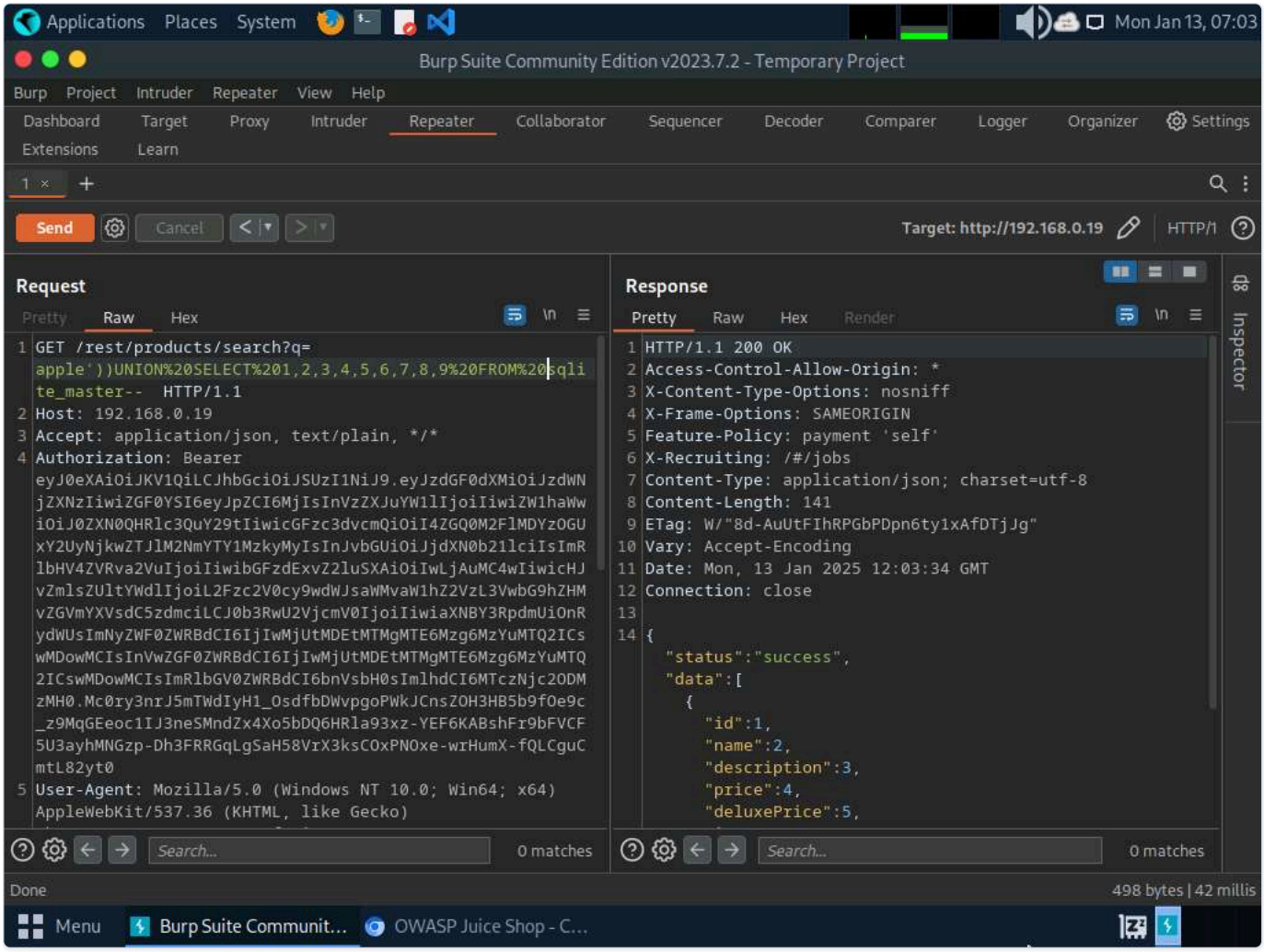
1. Test additional SQL syntax (such as appending `--` or `'))`) to observe backend query parsing behavior.



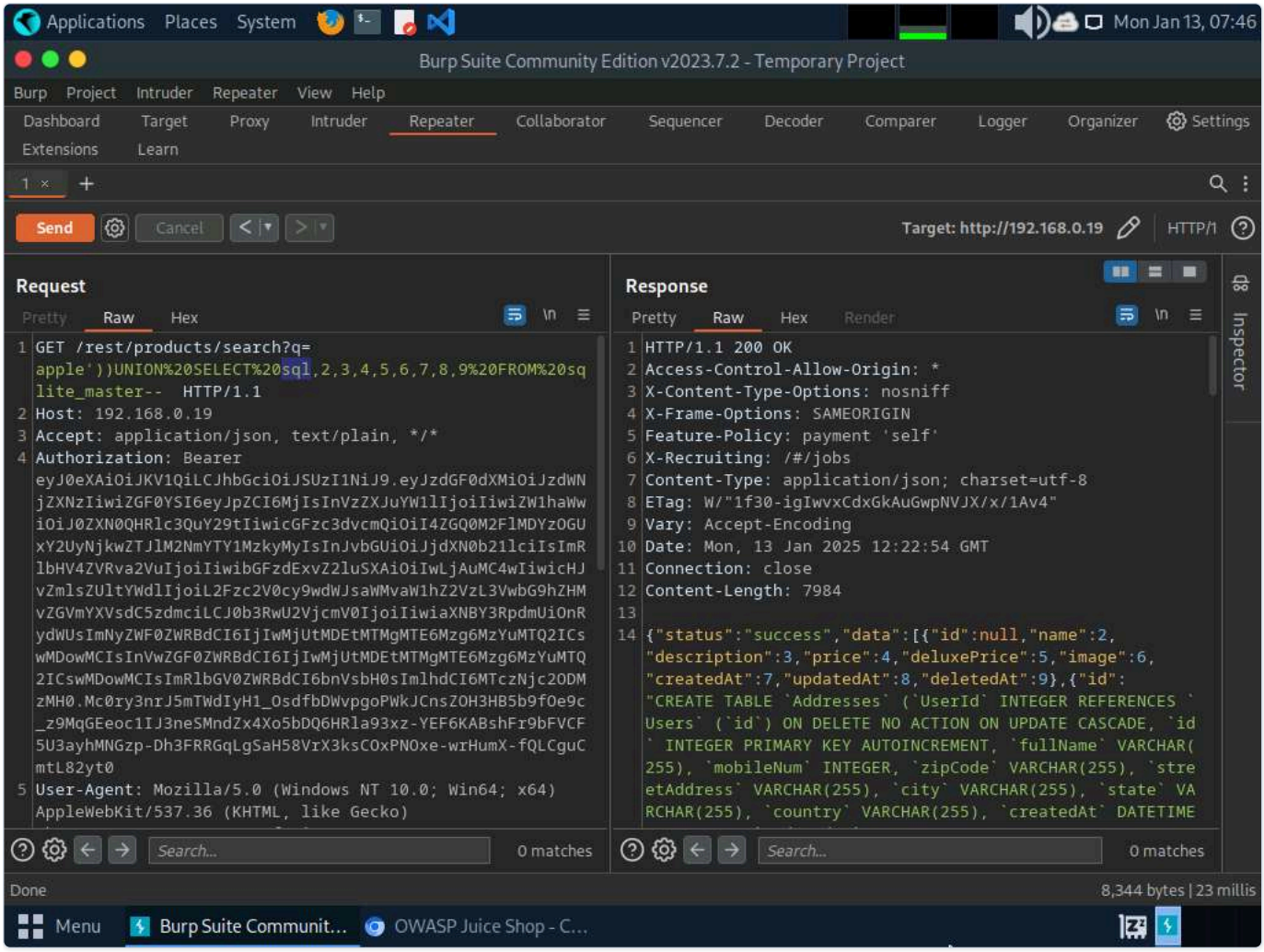
1. Attempt a `UNION SELECT` query to enumerate database structure:
   UNION%20SELECT%20*%20FROM%20sqlite_master--
   An error is returned because the number of selected columns does not match the original query column count.



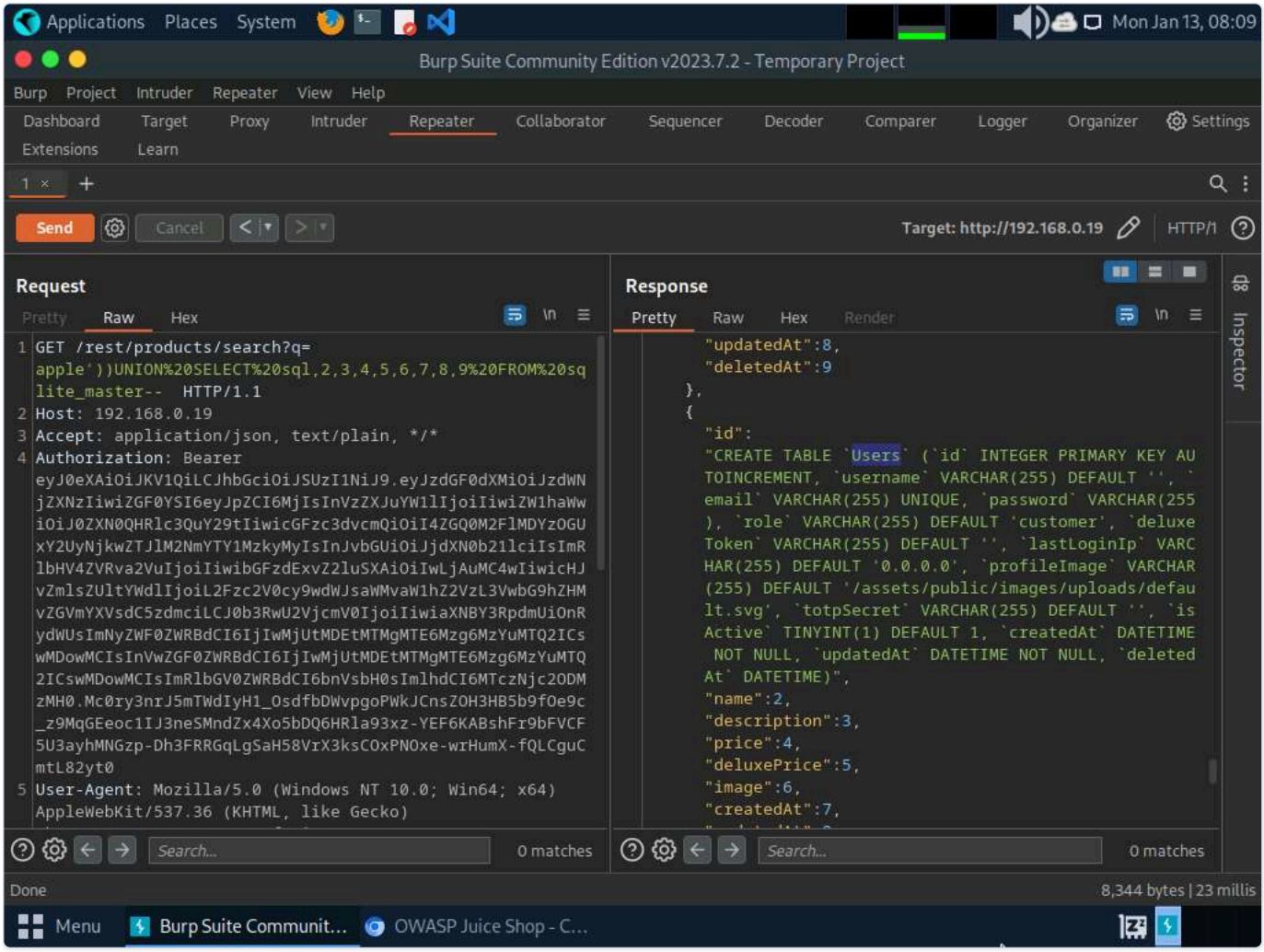
1. Incrementally add column positions to match the original query structure. Inject:
   UNION%20SELECT%201,2,3,4,5,6,7,8,9%20FROM%20sqlite_master--
   Click **Send**; the query succeeds, proving the backend query expects **9 columns**.



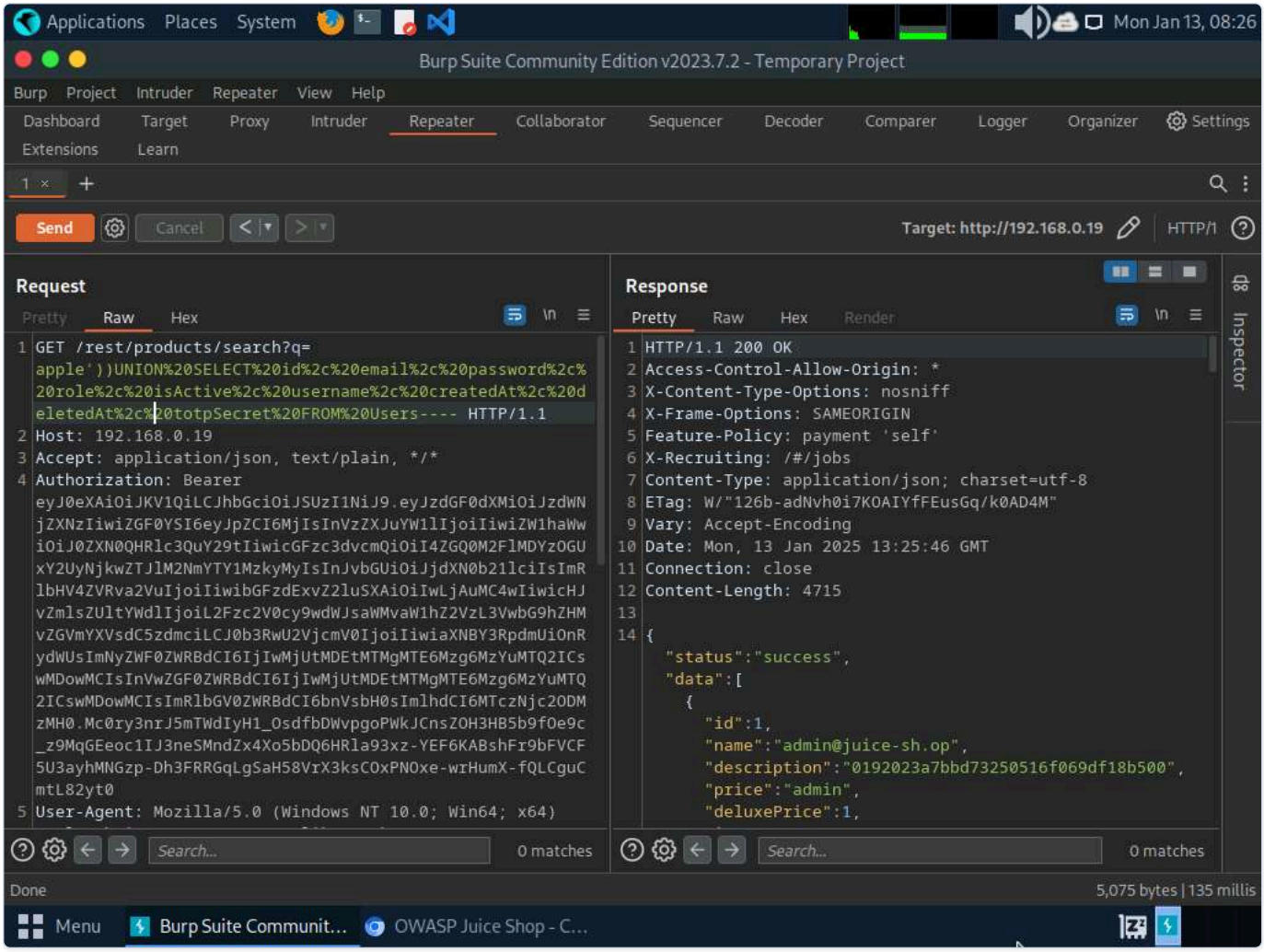
1. Replace position `1` with the column name **sql** to retrieve table creation definitions from SQLite master metadata:
   /rest/products/search?q=apple'))UNION%20SELECT%20sql,2,3,4,5,6,7,8,9%20FROM%20sqlite_master--



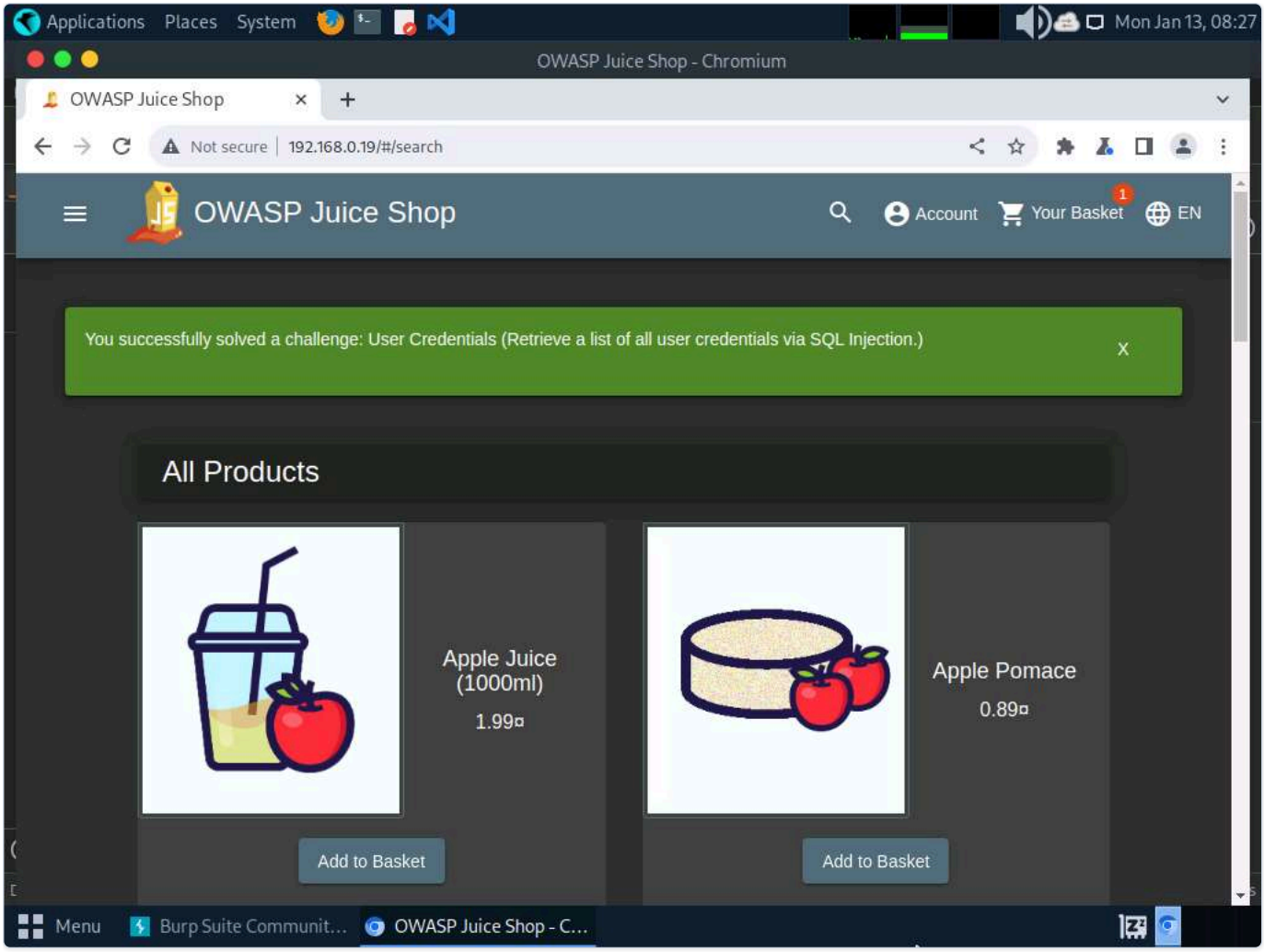
1. Inspect the response body to extract the schema definition for the **Users** table, identifying columns: `id`, `email`, `password`, `role`, `isActive`, `username`, `createdAt`, `deletedAt`, `totpSecret`.



1. Construct a UNION query to dump all columns from the **Users** table:
   UNION%20SELECT%20id%2c%20email%2c%20password%2c%20role%2c%20isActive%2c%20username%2c%20createdAt%2c%20deletedAt%2c%20totpSecret%20--
   Click **Send** to extract user accounts, roles, and hashed passwords.



1. Switch back to the Juice Shop browser window to confirm successful completion of the challenge.



This concludes the lab exercise.

IMPORTANT: ❓ **Question 5.11.1.1** Execute the OWASP Juice Shop web application on an Ubuntu machine. Using a Parrot Security system, conduct SQL injection attacks to retrieve the database schema and extract sensitive user credentials from the application hosted at `http://192.168.0.19`. Enter the request that was modified in this lab. [!SUCCESS] ✅ **Answer:** **GET /rest/products/search?q=**